

Contents

 C++ 三周入门手册	2
一、教学目标	2
学习定位	2
预期成果	2
推荐学习资料	3
二、教程安排	3
第一周安排 (Day 1-7)	4
第二周安排 (Day 8-14)	4
第三周安排 (Day 15-21)	5
三、预期成果	5
四、每日学习内容	5
项目结构准备	5
第一周：从 C 到 C++ ——语法进化与基本对象	7
Day 1: C++ 程序结构与输入输出	7
Day 2-3: 类与对象初步	8
Day 4-5: 构造函数进阶与对象生命周期	11
Day 6-7: 组合与对象成员	13
第二周：深入面向对象——继承与多态	16
Day 8-9: 继承机制与派生类	16
Day 10-11: 虚函数与多态	18
Day 12-13: 抽象类与接口设计	22
Day 14: 友元与运算符重载	26
第三周：项目化与 C++ 工程实践	29
Day 15-16: 文件与流操作	29
Day 17-18: 模块化与头文件管理	35
Day 19: 简单项目管理与 CMake 实践	40
Day 20-21: 综合项目实践 ①	42
Day 20-21.ex: 综合项目实践 ② - 控制台 Flappy Bird (OOP 版)	46
五、进阶扩展模块 (可选)	50
进阶模块概览	50
E1: 泛型编程 (Templates)	51
E2: 智能指针 (Smart Pointers)	52
E3: STL 容器与算法	53
E4: Lambda 表达式与函数对象	53
📖 进阶学习路线建议	54
六、学习建议与总结	55
6.1 学习建议	55

6.2 进阶方向	56
6.3 总结	57

C++ 三周入门手册

一、教学目标

本教程是《C 语言三周入门》的后继教程，专为已掌握 C 语言基础的学习者设计，旨在帮助大家在三周时间内从**面向过程编程**平滑过渡到**面向对象编程（OOP）**，系统掌握 C++ 的核心特性。

学习定位

- **前置知识**：已完成《C 语言三周入门》，熟悉 C 语言基本语法、函数、数组、指针与 Makefile 基本使用
- **学习重点**：理解面向对象三大特性（**封装、继承、多态**），掌握类与对象、模板与 STL 等核心概念
- **能力目标**：能够独立完成中小型面向对象项目的设计与实现（如学生成绩管理系统、控制台游戏等）
- **思维升级**：从“面向过程的函数组织”转变为“面向对象的类设计”，建立新的编程思维模式

预期成果

通过三周学习，你将能够：

- ☒ **理解 C++ 对 C 的扩展**：掌握 namespace、引用、iostream 等新特性
- ☒ **掌握类与对象**：能够设计类、封装数据与行为、理解构造/析构机制
- ☒ **运用继承与多态**：能够构建类层次结构、使用虚函数实现多态
- ☒ **组织工程化项目**：能够进行多文件编程、使用 CMake 构建完整项目
- ☒ **完成综合项目**：独立设计并实现一个面向对象的实用系统

【重要提示】

- 本教程**不能**替代系统的编程课程，仅作为从 C 到 C++ 的快速过渡指南
- 建议将本教程与《C 语言三周入门》对照学习，理解两种编程范式的差异与联系
- 所有小任务和综合项目均为**参考示例**，完成情况**不计入考核评分**

推荐学习资料

更系统的学习资料请参考以下资源：

在线教程：

- C++ 官方文档：权威的 C++ 参考手册【不适合入门，但是重要的参考资料】
- 菜鸟教程 C++ 教程：提供大量实例与清晰解释，从基础语法到高级特性，循序渐进引导学习。【适合新手入门，建议与其他资料结合使用】
- LearnCpp.com：优秀的英文 C++ 教程网站，讲解清晰详细。

书籍：

- 《C++ Primer(第 5 版)》：全面介绍 C++ 语言，从基础到高级，适合系统学习。【注】《C++ Primer Plus》是另一本不同的书籍，学习体验上不如 C++ Primer。
- 《Effective C++》：进阶必读，讲解 C++ 编程的最佳实践和常见陷阱。

视频课程：

- B 站“C++ 从入门到精通”系列视频：众多 UP 主制作的优质教程，例如黑马程序员的 C++ 教程（强烈推荐）
 - 可跳过前半部分与 C 语言重合部分的内容，重点关注从 01 程序的内存模型-内存四区-代码区。到 64 C++ 文件操作-二进制文件-读文件 的内容，后续模板和容器等相关的内容按需学习。
- 中国大学 MOOC：C++ 程序设计（北京大学郭炜老师主讲）

以上资源均适合有一定编程基础的学习者，能够帮助你系统掌握 C++ 编程的核心知识与技能。建议结合多种形式的学习资料，选一个你自己能接受学进去的作为主要学习材料，其他的作为辅助，理论与实践相结合，提升编程能力。

二、教程安排

教程共 21 天，分为三周进行。

- **第一周**：重点从 C 过渡到 C++，掌握类与对象、封装思想，理解 C++ 对 C 语言的扩展。
- **第二周**：重点学习 继承、多态、抽象类、运算符重载，深入理解面向对象三大特性。
- **第三周**：重点学习 文件操作、模块化编程、CMake 工程化构建，并完成综合项目“学生成绩管理系统”，扩展项目为控制台 Flappy Bird。

此外，教程还提供了 C++ 的部分扩展内容的学习大纲：

- E1：泛型编程 (Templates)
- E2：智能指针 (Smart Pointers)

- E3 : STL 容器与算法
- E4 : Lambda 表达式与函数对象

本教程的每天学习内容包括：

- **知识点**：当天需要掌握的核心概念与语法。
- **学习资料**：推荐的学习资源链接，基于**菜鸟教程**和 **C++ 官方文档**等资料，便于查阅。可以根据个人喜好选择其他资料。
- **应知应会 Checklist**：当天学习结束后需要掌握的技能点。
- **小任务**：通过编写代码完成的小练习，巩固当天所学内容。

【注】本教程提供的小任务和综合项目均为参考示例，学习者可根据自身兴趣和需求进行调整和扩展。**不强制完成**，完成情况**不计入最终考核评分**。

第一周安排 (Day 1-7)

天数	主题	知识点	小任务
D1	C++ 程序结构与输入输出	C++ 与 C 的区别、iostream、namespace std	编写 Hello World 程序，改写 C 程序
D2-D3	类与对象初步	class 定义、访问控制、构造/析构函数	定义 Student 类并打印学生信息
D4-D5	构造函数进阶与对象生命周期	构造函数重载、拷贝构造函数、深拷贝 vs 浅拷贝	实现带拷贝构造的 Book 类
D6-D7	组合与对象成员	has-a 关系、成员初始化列表、对象成员构造顺序	创建 Student 与 Score 类的组合关系

第二周安排 (Day 8-14)

天数	主题	知识点	小任务
D8-D9	继承机制与派生类	is-a 关系、三种继承方式、派生类构造与析构	创建 Person 基类与子类
D10-D11	虚函数与多态	virtual、override、动态绑定、虚析构函数	用多态实现不同类型用户信息打印
D12-D13	抽象类与接口设计	纯虚函数、抽象基类、接口分离原则	设计 Shape 接口，派生 Circle 与 Rectangle
D14	友元与运算符重载	friend 函数与类、运算符重载 (+, ==, <)	为 Complex 类重载加法和输出运算符

第三周安排 (Day 15–21)

天数	主题	知识点	小任务
D15–D16	文件与流操作	fstream、文本/二进制文件读写	实现学生信息存档系统
D17–D18	模块化与头文件管理	.h/.cpp 分离、头文件防卫	分文件实现学生管理系统
D19	CMake 实践	CMakeLists.txt、跨平台构建	使用 CMake 构建学生成绩系统
D20–D21	综合项目实践	面向对象设计、文件存储、用户交互	完整学生成绩管理系统或 Flappy Bird

三、预期成果

- 同学们能独立编写 C++ 面向对象程序
- 能熟练使用类、继承、多态等核心特性
- 理解 C++ 的内存管理与对象生命周期
- 掌握多文件工程组织与 CMake 构建
- 完成一个完整的面向对象项目

四、每日学习内容

项目结构准备

在开始正式学习之前，我们需要先准备好项目的基本目录结构，方便每天的代码和总结文档进行管理。

【注】本项目结构仅为建议，学习者可根据个人习惯进行调整，但建议保持清晰有序，便于后续复习和查阅。

操作步骤

1. 打开你的工作目录（例如 D:\my-cpp-project 或 ~/my-cpp-project）。
2. 在其中新建一个文件夹：

```
my-cpp-project/  
└─ cpp 学习/
```

3. 在 `cpp 学习` 文件夹下，建立 `Day 1 ~ Day 21` 共 **21 个子文件夹**，用于存放每天的代码和练习：

```
cpp 学习/  
├─ Day1/  
├─ Day2/  
├─ ...  
└─ Day21/
```

文件组织规范

- 每个 `DayX/` 文件夹下，至少包含：
 - 当天的小任务代码文件（例如 `main.cpp`、`task.cpp` 等）。
 - 如果有总结或笔记，推荐使用 **Markdown 文件**（例如 `notes.md`）来记录当天的学习要点。

例如，`Day1/` 文件夹可能长这样：

```
Day1/  
├─ main.cpp      # 小任务代码  
└─ notes.md      # 知识点总结
```

提交至 Git 与 GitHub

1. 将 `my-cpp-project` 文件夹初始化为一个 Git 仓库：

```
cd my-cpp-project  
git init
```

2. 每天学习结束后，将当天的代码和笔记提交到 Git 仓库：

```
git add .  
git commit -m "完成 DayX 学习任务"
```

3. 如果有远程仓库（例如 GitHub），可以将本地仓库与远程仓库关联，并推送代码：

```
git remote add origin <你的远程仓库地址>  
git push -u origin main
```

【注】推荐将每日的进度提交到 Git 仓库和 GitHub，养成良好的版本管理习惯，便于后续复习和查阅。

推荐做法

- **代码文件**：将每天的小任务实现写在 `.cpp` 文件中，如果涉及类定义可拆分成 `.h` 和 `.cpp` 文件。
- **笔记文件**：使用 Markdown 简洁总结：
 - 当天学习的语法/概念
 - 小任务的实现思路
 - 遇到的问题与解决办法

这样到第 21 天结束时，你将拥有一个完整的 **学习代码库 + 学习日志**，方便复习与回顾。

第一周：从 C 到 C++ ——语法进化与基本对象

Day 1: C++ 程序结构与输入输出

📖 1.1 基本知识点

- C++ 程序的基本结构
- 使用 `iostream` 进行输入输出
- 命名空间 `namespace std`
- C 与 C++ 的主要语法差异
- 编译 C++ 程序的方法

🔗 1.2 学习资料

- 菜鸟教程：
 - C++ 教程
 - C++ 基本语法
 - C++ 基本输入输出
- LearnCpp: Introduction to C++

☑ 1.3 应知应会 Checklist

- ☐ 能编写并编译 C++ 程序
- ☐ 理解 `#include <iostream>` 的作用
- ☐ 掌握 `cin` 和 `cout` 的使用
- ☐ 了解 `using namespace std` 的含义
- ☐ 知道 C++ 与 C 的主要区别

📝 1.4 小任务

1. 编写第一个 C++ 程序，输出 "Hello, C++!"

2. 改写一个 C 语言的输入输出程序为 C++ 版本

```
// hello.cpp
#include <iostream>
using namespace std;

int main() {
    cout << "Hello, C++!" << endl;
    return 0;
}
```

```
// input_output.cpp
#include <iostream>
using namespace std;

int main() {
    string name;
    int age;

    cout << " 请输入你的名字: ";
    cin >> name;
    cout << " 请输入你的年龄: ";
    cin >> age;

    cout << " 你好, " << name << "! 你今年 " << age << " 岁。" << endl;

    return 0;
}
```

编译运行:

```
g++ hello.cpp -o hello
./hello

g++ input_output.cpp -o input_output
./input_output
```

Day 2-3: 类与对象初步

■ 2.1 基本知识点

- 类的定义与声明
- 访问控制: public、private、protected
- 成员变量与成员函数
- 构造函数与析构函数
- 对象的创建与使用

🔗 2.2 学习资料

- 菜鸟教程:
 - C++ 类 & 对象
 - C++ 类访问修饰符
 - C++ 类构造函数 & 析构函数
- LearnCpp: Introduction to classes

☑ 2.3 应知应会 Checklist

- ☐ 能定义简单的类
- ☐ 理解 public 和 private 的区别
- ☐ 会编写构造函数和析构函数
- ☐ 能创建对象并调用成员函数
- ☐ 理解封装的概念

📁 2.4 小任务 定义一个 Student 类, 包含姓名、学号、成绩等属性, 提供方法显示学生信息。

```
// student.cpp
#include <iostream>
#include <string>
using namespace std;

class Student {
private:
    string name;
    string id;
    double score;

public:
    // 构造函数
    Student(string n, string i, double s) {
        name = n;
```

```

        id = i;
        score = s;
        cout << " 学生对象创建: " << name << endl;
    }

// 析构函数
~Student() {
    cout << " 学生对象销毁: " << name << endl;
}

// 成员函数
void showInfo() {
    cout << " 姓名: " << name << endl;
    cout << " 学号: " << id << endl;
    cout << " 成绩: " << score << endl;
}

void setScore(double s) {
    if (s >= 0 && s <= 100) {
        score = s;
    } else {
        cout << " 成绩无效! " << endl;
    }
}

double getScore() {
    return score;
}
};

int main() {
    Student stu1(" 张三", "2023001", 85.5);
    stu1.showInfo();

    cout << "\n修改成绩..." << endl;
    stu1.setScore(90.0);
    cout << " 新成绩: " << stu1.getScore() << endl;
}

```

```
return 0;  
}
```

Day 4-5: 构造函数进阶与对象生命周期

▣ 4.1 基本知识点

- 构造函数重载
- 默认构造函数
- 拷贝构造函数
- 赋值操作符重载
- 析构函数的调用时机
- 对象的生命周期

🔗 4.2 学习资料

- 菜鸟教程：
 - C++ 拷贝构造函数
 - C++ 友元函数
- LearnCpp: Constructor member initializer lists

☑ 4.3 应知应会 Checklist

- ☐ 理解构造函数重载
- ☐ 会编写拷贝构造函数
- ☐ 理解深拷贝与浅拷贝的区别
- ☐ 了解对象的创建和销毁过程
- ☐ 掌握成员初始化列表

📝 4.4 小任务 实现一个 Book 类，包含书名、作者、价格，并实现拷贝构造函数。

```
// book.cpp  
#include <iostream>  
#include <string>  
using namespace std;  
  
class Book {  
private:  
    string title;
```

```

string author;
double price;
int* pages; // 动态分配的页数

public:
    // 默认构造函数
    Book() : title(" 未知"), author(" 未知"), price(0.0) {
        pages = new int(0);
        cout << " 调用默认构造函数" << endl;
    }

    // 带参数的构造函数
    Book(string t, string a, double p, int pg)
        : title(t), author(a), price(p) {
        pages = new int(pg);
        cout << " 创建书籍: " << title << endl;
    }

    // 拷贝构造函数 (深拷贝)
    Book(const Book& other) {
        title = other.title;
        author = other.author;
        price = other.price;
        pages = new int(*other.pages); // 深拷贝
        cout << " 拷贝构造: " << title << endl;
    }

    // 析构函数
    ~Book() {
        delete pages;
        cout << " 销毁书籍: " << title << endl;
    }

    void showInfo() const {
        cout << " 《" << title << "》 " << endl;
        cout << " 作者: " << author << endl;
        cout << " 价格: ¥" << price << endl;
        cout << " 页数: " << *pages << endl;
    }

```

```

    }
};

int main() {
    Book book1("C++ Primer", "Stanley B. Lippman", 99.0, 850);
    book1.showInfo();

    cout << "\n创建副本..." << endl;
    Book book2 = book1; // 调用拷贝构造函数
    book2.showInfo();

    return 0;
}

```

Day 6-7: 组合与对象成员

▣ 6.1 基本知识点

- 类的组合 (has-a 关系)
- 对象成员
- 成员初始化列表
- 对象成员的构造顺序
- 嵌套类

🔗 6.2 学习资料

- 菜鸟教程: C++ 类 & 对象
- 相关博客: C++ 对象组合
- LearnCpp: Class composition

☑ 6.3 应知应会 Checklist

- ☐ 理解类的组合关系
- ☐ 会使用成员初始化列表
- ☐ 理解对象成员的构造和析构顺序
- ☐ 能设计包含多个类的系统

📁 6.4 小任务 创建 Score 类和 Student 类, Student 包含多个 Score 对象。

```

// student_score.cpp
#include <iostream>
#include <string>
using namespace std;

class Score {
private:
    string subject;
    double value;

public:
    Score(string s = " 未知科目", double v = 0.0)
        : subject(s), value(v) {
        cout << " 创建成绩对象: " << subject << endl;
    }

    ~Score() {
        cout << " 销毁成绩对象: " << subject << endl;
    }

    void setScore(double v) {
        if (v >= 0 && v <= 100) {
            value = v;
        }
    }

    void showScore() const {
        cout << subject << ": " << value << " 分" << endl;
    }
};

class Student {
private:
    string name;
    string id;
    Score mathScore;
    Score englishScore;
    Score cppScore;

```

```

public:
    // 使用成员初始化列表
    Student(string n, string i, double math, double eng, double cpp)
        : name(n), id(i),
          mathScore(" 数学", math),
          englishScore(" 英语", eng),
          cppScore("C++", cpp) {
        cout << " 创建学生对象: " << name << endl;
    }

    ~Student() {
        cout << " 销毁学生对象: " << name << endl;
    }

    void showInfo() const {
        cout << "\n===== 学生信息 =====" << endl;
        cout << " 姓名: " << name << endl;
        cout << " 学号: " << id << endl;
        cout << " 成绩信息:" << endl;
        mathScore.showScore();
        englishScore.showScore();
        cppScore.showScore();
        cout << "=====\\n" << endl;
    }
};

int main() {
    Student stu(" 李四", "2023002", 88.0, 92.0, 95.0);
    stu.showInfo();

    return 0;
}

```

第二周：深入面向对象——继承与多态

Day 8-9：继承机制与派生类

📖 8.1 基本知识点

- 继承的概念与语法
- public、protected、private 继承
- 基类与派生类
- 继承中的构造函数与析构函数
- 访问权限的变化
- is-a 关系

🔗 8.2 学习资料

- 菜鸟教程：C++ 继承
- LearnCpp: Basic inheritance

☑ 8.3 应知应会 Checklist

- ☐ 理解继承的概念与用途
- ☐ 掌握三种继承方式的区别
- ☐ 理解派生类的构造和析构顺序
- ☐ 能设计基类与派生类的关系
- ☐ 理解代码复用的意义

📝 8.4 小任务 创建 Person 基类，派生出 Teacher 和 Student 子类。

```
// inheritance.cpp
#include <iostream>
#include <string>
using namespace std;

// 基类
class Person {
protected:
    string name;
    int age;

public:
    Person(string n, int a) : name(n), age(a) {
        cout << "Person 构造函数: " << name << endl;
    }
};
```

```

    }

    ~Person() {
        cout << "Person 析构函数: " << name << endl;
    }

    void showBasicInfo() const {
        cout << " 姓名: " << name << endl;
        cout << " 年龄: " << age << endl;
    }
};

// 派生类: Teacher
class Teacher : public Person {
private:
    string subject;
    int workYears;

public:
    Teacher(string n, int a, string s, int y)
        : Person(n, a), subject(s), workYears(y) {
        cout << "Teacher 构造函数: " << name << endl;
    }

    ~Teacher() {
        cout << "Teacher 析构函数: " << name << endl;
    }

    void showInfo() const {
        showBasicInfo();
        cout << " 任教科目: " << subject << endl;
        cout << " 工作年限: " << workYears << " 年" << endl;
    }
};

// 派生类: Student
class Student : public Person {
private:

```

```

    string studentId;
    double gpa;

public:
    Student(string n, int a, string id, double g)
        : Person(n, a), studentId(id), gpa(g) {
        cout << "Student 构造函数: " << name << endl;
    }

    ~Student() {
        cout << "Student 析构函数: " << name << endl;
    }

    void showInfo() const {
        showBasicInfo();
        cout << " 学号: " << studentId << endl;
        cout << "GPA: " << gpa << endl;
    }
};

int main() {
    cout << "===== 创建教师对象 =====" << endl;
    Teacher teacher(" 王老师", 35, " 数学", 10);
    teacher.showInfo();

    cout << "\n===== 创建学生对象 =====" << endl;
    Student student(" 张三", 20, "2023001", 3.8);
    student.showInfo();

    cout << "\n===== 对象销毁 =====" << endl;
    return 0;
}

```

Day 10-11: 虚函数与多态

▣ 10.1 基本知识点

- 虚函数 virtual

- 函数重写 (override)
- 动态绑定与静态绑定
- 虚析构函数
- 多态的概念与应用
- 基类指针与派生类对象

🔗 10.2 学习资料

- 菜鸟教程：
 - C++ 多态
 - C++ 虚函数
- LearnCpp: Virtual functions

☑ 10.3 应知应会 Checklist

- ☐ 理解虚函数的作用
- ☐ 掌握多态的实现机制
- ☐ 理解动态绑定与静态绑定的区别
- ☐ 知道何时使用虚析构函数
- ☐ 能用多态实现灵活的程序设计

📁 10.4 小任务 用多态实现不同类型用户信息的打印。

```
// polymorphism.cpp
#include <iostream>
#include <string>
#include <vector>
using namespace std;

// 基类
class Person {
protected:
    string name;
    int age;

public:
    Person(string n, int a) : name(n), age(a) {}

    // 虚析构函数
    virtual ~Person() {
```

```

        cout << "~Person()" << endl;
    }

    // 虚函数
    virtual void showInfo() const {
        cout << " 姓名: " << name << ", 年龄: " << age << endl;
    }

    virtual void getRole() const {
        cout << " 角色: 普通人" << endl;
    }
};

class Teacher : public Person {
private:
    string subject;

public:
    Teacher(string n, int a, string s)
        : Person(n, a), subject(s) {}

    ~Teacher() override {
        cout << "~Teacher()" << endl;
    }

    // 重写虚函数
    void showInfo() const override {
        Person::showInfo();
        cout << " 任教科目: " << subject << endl;
    }

    void getRole() const override {
        cout << " 角色: 教师" << endl;
    }
};

class Student : public Person {
private:

```

```

    string studentId;

public:
    Student(string n, int a, string id)
        : Person(n, a), studentId(id) {}

    ~Student() override {
        cout << "~Student()" << endl;
    }

    void showInfo() const override {
        Person::showInfo();
        cout << " 学号: " << studentId << endl;
    }

    void getRole() const override {
        cout << " 角色: 学生" << endl;
    }
};

// 多态函数
void printPersonInfo(const Person& p) {
    p.getRole();
    p.showInfo();
    cout << "-----" << endl;
}

int main() {
    Teacher teacher(" 李老师", 40, " 物理");
    Student student(" 王小明", 19, "2023002");
    Person person(" 路人甲", 25);

    cout << "===== 使用多态 =====" << endl;
    printPersonInfo(teacher);
    printPersonInfo(student);
    printPersonInfo(person);

    cout << "\n===== 使用基类指针 =====" << endl;
}

```

```
Person* p1 = &teacher;
Person* p2 = &student;

p1->showInfo();
cout << endl;
p2->showInfo();

return 0;
}
```

Day 12-13: 抽象类与接口设计

📖 12.1 基本知识点

- 纯虚函数 = 0
- 抽象基类
- 接口的概念
- 接口设计原则
- 抽象类的应用场景

🔗 12.2 学习资料

- 菜鸟教程: C++ 接口 (抽象类)
- LearnCpp: Pure virtual functions and abstract base classes

☑ 12.3 应知应会 Checklist

- ☐ 理解纯虚函数的作用
- ☐ 知道抽象类不能实例化
- ☐ 能设计抽象基类作为接口
- ☐ 理解接口分离原则
- ☐ 能用抽象类构建可扩展的系统

📁 12.4 小任务 设计一个 Shape 抽象类接口, 派生 Circle 和 Rectangle 类。

```
// abstract_shape.cpp
#include <iostream>
#include <cmath>
#include <vector>
```

```

using namespace std;

// 抽象基类 (接口)
class Shape {
protected:
    string name;

public:
    Shape(string n) : name(n) {}
    virtual ~Shape() {}

    // 纯虚函数
    virtual double getArea() const = 0;
    virtual double getPerimeter() const = 0;
    virtual void display() const = 0;

    string getName() const {
        return name;
    }
};

// 派生类: 圆形
class Circle : public Shape {
private:
    double radius;

public:
    Circle(double r) : Shape(" 圆形"), radius(r) {}

    double getArea() const override {
        return M_PI * radius * radius;
    }

    double getPerimeter() const override {
        return 2 * M_PI * radius;
    }

    void display() const override {

```

```

        cout << " 形状: " << name << endl;
        cout << " 半径: " << radius << endl;
        cout << " 面积: " << getArea() << endl;
        cout << " 周长: " << getPerimeter() << endl;
    }
};

// 派生类: 矩形
class Rectangle : public Shape {
private:
    double width;
    double height;

public:
    Rectangle(double w, double h)
        : Shape(" 矩形"), width(w), height(h) {}

    double getArea() const override {
        return width * height;
    }

    double getPerimeter() const override {
        return 2 * (width + height);
    }

    void display() const override {
        cout << " 形状: " << name << endl;
        cout << " 宽度: " << width << ", 高度: " << height << endl;
        cout << " 面积: " << getArea() << endl;
        cout << " 周长: " << getPerimeter() << endl;
    }
};

// 派生类: 三角形
class Triangle : public Shape {
private:
    double a, b, c;

```

```

public:
    Triangle(double side_a, double side_b, double side_c)
        : Shape(" 三角形"), a(side_a), b(side_b), c(side_c) {}

    double getArea() const override {
        // 使用海伦公式
        double s = (a + b + c) / 2;
        return sqrt(s * (s - a) * (s - b) * (s - c));
    }

    double getPerimeter() const override {
        return a + b + c;
    }

    void display() const override {
        cout << " 形状: " << name << endl;
        cout << " 三边: " << a << ", " << b << ", " << c << endl;
        cout << " 面积: " << getArea() << endl;
        cout << " 周长: " << getPerimeter() << endl;
    }
};

int main() {
    // 无法实例化抽象类
    // Shape shape(" 测试"); // 错误!

    vector<Shape*> shapes;
    shapes.push_back(new Circle(5.0));
    shapes.push_back(new Rectangle(4.0, 6.0));
    shapes.push_back(new Triangle(3.0, 4.0, 5.0));

    cout << "===== 所有图形信息 =====" << endl;
    for (size_t i = 0; i < shapes.size(); i++) {
        shapes[i]->display();
        cout << "-----" << endl;
    }

    // 清理内存

```

```
    for (Shape* s : shapes) {  
        delete s;  
    }  
  
    return 0;  
}
```

Day 14: 友元与运算符重载

📖 14.1 基本知识点

- 友元函数 friend
- 友元类
- 运算符重载的概念
- 常用运算符重载：+、-、*、==、<<、>>
- 成员函数形式的运算符重载
- 友元函数形式的运算符重载

🔗 14.2 学习资料

- 菜鸟教程：
 - C++ 友元函数
 - C++ 运算符重载
- LearnCpp: Operator overloading

☑ 14.3 应知应会 Checklist

- ☐ 理解友元的概念与用途
- ☐ 掌握运算符重载的语法
- ☐ 能重载常用运算符
- ☐ 知道何时使用成员函数或友元函数重载
- ☐ 能让类的使用更加自然

📝 14.4 小任务 为 Complex 复数类重载加法、减法和输出运算符。

```
// complex.cpp  
#include <iostream>  
using namespace std;
```

```

class Complex {
private:
    double real;
    double imag;

public:
    Complex(double r = 0.0, double i = 0.0) : real(r), imag(i) {}

    // 运算符重载 (成员函数形式)
    Complex operator+(const Complex& other) const {
        return Complex(real + other.real, imag + other.imag);
    }

    Complex operator-(const Complex& other) const {
        return Complex(real - other.real, imag - other.imag);
    }

    Complex operator*(const Complex& other) const {
        return Complex(
            real * other.real - imag * other.imag,
            real * other.imag + imag * other.real
        );
    }

    bool operator==(const Complex& other) const {
        return (real == other.real) && (imag == other.imag);
    }

    // 友元函数: 输出运算符重载
    friend ostream& operator<<(ostream& os, const Complex& c);

    // 友元函数: 输入运算符重载
    friend istream& operator>>(istream& is, Complex& c);
};

// 友元函数实现
ostream& operator<<(ostream& os, const Complex& c) {
    os << c.real;

```

```

    if (c.imag >= 0) {
        os << " + " << c.imag << "i";
    } else {
        os << " - " << -c.imag << "i";
    }
    return os;
}

istream& operator>>(istream& is, Complex& c) {
    cout << " 请输入实部: ";
    is >> c.real;
    cout << " 请输入虚部: ";
    is >> c.imag;
    return is;
}

int main() {
    Complex c1(3.0, 4.0);
    Complex c2(1.0, 2.0);

    cout << "c1 = " << c1 << endl;
    cout << "c2 = " << c2 << endl;

    Complex c3 = c1 + c2;
    cout << "c1 + c2 = " << c3 << endl;

    Complex c4 = c1 - c2;
    cout << "c1 - c2 = " << c4 << endl;

    Complex c5 = c1 * c2;
    cout << "c1 * c2 = " << c5 << endl;

    if (c1 == c2) {
        cout << "c1 == c2" << endl;
    } else {
        cout << "c1 != c2" << endl;
    }
}

```

```
cout << "\n输入一个复数: " << endl;
Complex c6;
cin >> c6;
cout << " 你输入的复数是: " << c6 << endl;

return 0;
}
```

第三周：项目化与 C++ 工程实践

Day 15-16：文件与流操作

📖 15.1 基本知识点

- 文件流 fstream
- 文本文件读写
- 二进制文件读写
- 文件打开模式
- 数据持久化
- 文件格式设计

🔗 15.2 学习资料

- 菜鸟教程：C++ 文件和流
- LearnCpp: File I/O

☑ 15.3 应知应会 Checklist

- ☐ 掌握文件的打开与关闭
- ☐ 能读写文本文件
- ☐ 理解二进制文件的读写方式
- ☐ 能设计合理的文件格式
- ☐ 理解数据持久化的意义

🔧 15.4 小任务 实现一个简单的学生信息存档系统，支持保存和加载。

```
// file_io.cpp
#include <iostream>
#include <fstream>
#include <string>
```

```

#include <vector>
using namespace std;

class Student {
private:
    string name;
    string id;
    double score;

public:
    Student(string n = "", string i = "", double s = 0.0)
        : name(n), id(i), score(s) {}

    void display() const {
        cout << " 姓名: " << name << ", 学号: " << id
            << ", 成绩: " << score << endl;
    }

    // 保存到文件
    void saveToFile(ofstream& ofs) const {
        ofs << name << endl;
        ofs << id << endl;
        ofs << score << endl;
    }

    // 从文件加载
    void loadFromFile(ifstream& ifs) {
        getline(ifs, name);
        getline(ifs, id);
        ifs >> score;
        ifs.ignore(); // 忽略换行符
    }

    // 二进制保存
    void saveBinary(ofstream& ofs) const {
        size_t nameLen = name.length();
        ofs.write((char*)&nameLen, sizeof(nameLen));
        ofs.write(name.c_str(), nameLen);
    }

```

```

        size_t idLen = id.length();
        ofs.write((char*)&idLen, sizeof(idLen));
        ofs.write(id.c_str(), idLen);

        ofs.write((char*)&score, sizeof(score));
    }

    // 二进制加载
    void loadBinary(istream& ifs) {
        size_t nameLen;
        ifs.read((char*)&nameLen, sizeof(nameLen));
        name.resize(nameLen);
        ifs.read(&name[0], nameLen);

        size_t idLen;
        ifs.read((char*)&idLen, sizeof(idLen));
        id.resize(idLen);
        ifs.read(&id[0], idLen);

        ifs.read((char*)&score, sizeof(score));
    }
};

class StudentManager {
private:
    vector<Student> students;

public:
    void addStudent(const Student& stu) {
        students.push_back(stu);
    }

    void displayAll() const {
        cout << "\n===== 所有学生信息 =====" << endl;
        for (const auto& stu : students) {
            stu.display();
        }
        cout << "=====\n" << endl;
    }
};

```

```

}

// 保存到文本文件
bool saveToTextFile(const string& filename) {
    ofstream ofs(filename);
    if (!ofs) {
        cout << " 文件打开失败! " << endl;
        return false;
    }

    ofs << students.size() << endl;
    for (const auto& stu : students) {
        stu.saveToFile(ofs);
    }

    ofs.close();
    cout << " 数据已保存到: " << filename << endl;
    return true;
}

// 从文本文件加载
bool loadFromTextFile(const string& filename) {
    ifstream ifs(filename);
    if (!ifs) {
        cout << " 文件打开失败! " << endl;
        return false;
    }

    students.clear();
    size_t count;
    ifs >> count;
    ifs.ignore();

    for (size_t i = 0; i < count; i++) {
        Student stu;
        stu.loadFromFile(ifs);
        students.push_back(stu);
    }
}

```

```

        ifs.close();
        cout << " 数据已从 " << filename << " 加载" << endl;
        return true;
    }

    // 保存到二进制文件
    bool saveToBinaryFile(const string& filename) {
        ofstream ofs(filename, ios::binary);
        if (!ofs) {
            cout << " 文件打开失败! " << endl;
            return false;
        }

        size_t count = students.size();
        ofs.write((char*)&count, sizeof(count));

        for (const auto& stu : students) {
            stu.saveBinary(ofs);
        }

        ofs.close();
        cout << " 数据已保存到二进制文件: " << filename << endl;
        return true;
    }

    // 从二进制文件加载
    bool loadFromBinaryFile(const string& filename) {
        ifstream ifs(filename, ios::binary);
        if (!ifs) {
            cout << " 文件打开失败! " << endl;
            return false;
        }

        students.clear();
        size_t count;
        ifs.read((char*)&count, sizeof(count));

        for (size_t i = 0; i < count; i++) {

```

```

        Student stu;
        stu.loadBinary(ifs);
        students.push_back(stu);
    }

    ifs.close();
    cout << " 数据已从二进制文件 " << filename << " 加载" << endl;
    return true;
}
};

int main() {
    StudentManager manager;

    manager.addStudent(Student(" 张三", "2023001", 85.5));
    manager.addStudent(Student(" 李四", "2023002", 92.0));
    manager.addStudent(Student(" 王五", "2023003", 78.5));

    manager.displayAll();

    // 保存到文本文件
    manager.saveToTextFile("students.txt");

    // 保存到二进制文件
    manager.saveToBinaryFile("students.dat");

    cout << "\n清空数据并重新加载..." << endl;
    StudentManager newManager;

    // 从文本文件加载
    newManager.loadFromTextFile("students.txt");
    newManager.displayAll();

    // 从二进制文件加载
    newManager.loadFromBinaryFile("students.dat");
    newManager.displayAll();

    return 0;
}

```

}

Day 17-18: 模块化与头文件管理

17.1 基本知识点

- .h 头文件与 .cpp 源文件的分离
- 头文件防卫 (Header Guards)
- #ifndef / #define / #endif
- #pragma once
- 多文件工程的组织
- 声明与定义的分离
- 编译与链接过程

17.2 学习资料

- 菜鸟教程: C++ 预处理器
- LearnCpp: Header files

17.3 应知应会 Checklist

- ☐ 理解头文件的作用
- ☐ 掌握头文件防卫的两种方式
- ☐ 能将类拆分到 .h 和 .cpp 文件
- ☐ 理解编译与链接的过程
- ☐ 能组织多文件项目

17.4 小任务 将学生系统拆分为多个文件。

项目结构:

```
student_system/  
├── student.h  
├── student.cpp  
├── student_manager.h  
├── student_manager.cpp  
└── main.cpp
```

student.h:

```

// student.h
#ifndef STUDENT_H
#define STUDENT_H

#include <string>
#include <fstream>
using namespace std;

class Student {
private:
    string name;
    string id;
    double score;

public:
    Student(string n = "", string i = "", double s = 0.0);

    void display() const;
    void saveToFile(ofstream& ofs) const;
    void loadFromFile(ifstream& ifs);

    string getName() const { return name; }
    string getId() const { return id; }
    double getScore() const { return score; }
};

#endif // STUDENT_H

```

student.cpp:

```

// student.cpp
#include "student.h"
#include <iostream>

Student::Student(string n, string i, double s)
    : name(n), id(i), score(s) {}

void Student::display() const {
    cout << " 姓名: " << name << ", 学号: " << id

```

```

        << " , 成绩: " << score << endl;
    }

void Student::saveToFile(ofstream& ofs) const {
    ofs << name << endl;
    ofs << id << endl;
    ofs << score << endl;
}

void Student::loadFromFile(istream& ifs) {
    getline(ifs, name);
    getline(ifs, id);
    ifs >> score;
    ifs.ignore();
}

```

student_manager.h:

```

// student_manager.h
#ifndef STUDENT_MANAGER_H
#define STUDENT_MANAGER_H

#include "student.h"
#include <vector>
#include <string>

class StudentManager {
private:
    vector<Student> students;

public:
    void addStudent(const Student& stu);
    void displayAll() const;
    bool saveToFile(const string& filename);
    bool loadFromFile(const string& filename);
    void searchByName(const string& name) const;
};

#endif // STUDENT_MANAGER_H

```

student_manager.cpp:

```
// student_manager.cpp
#include "student_manager.h"
#include <iostream>

void StudentManager::addStudent(const Student& stu) {
    students.push_back(stu);
}

void StudentManager::displayAll() const {
    cout << "\n===== 所有学生信息 =====" << endl;
    for (const auto& stu : students) {
        stu.display();
    }
    cout << "=====\n" << endl;
}

bool StudentManager::saveToFile(const string& filename) {
    ofstream ofs(filename);
    if (!ofs) {
        cout << " 文件打开失败! " << endl;
        return false;
    }

    ofs << students.size() << endl;
    for (const auto& stu : students) {
        stu.saveToFile(ofs);
    }

    ofs.close();
    cout << " 数据已保存到: " << filename << endl;
    return true;
}

bool StudentManager::loadFromFile(const string& filename) {
    ifstream ifs(filename);
    if (!ifs) {
        cout << " 文件打开失败! " << endl;
    }
}
```

```

        return false;
    }

    students.clear();
    size_t count;
    ifs >> count;
    ifs.ignore();

    for (size_t i = 0; i < count; i++) {
        Student stu;
        stu.loadFromFile(ifs);
        students.push_back(stu);
    }

    ifs.close();
    cout << " 数据已加载" << endl;
    return true;
}

void StudentManager::searchByName(const string& name) const {
    cout << "\n搜索结果: " << endl;
    bool found = false;
    for (const auto& stu : students) {
        if (stu.getName() == name) {
            stu.display();
            found = true;
        }
    }
    if (!found) {
        cout << " 未找到学生: " << name << endl;
    }
}
}

```

main.cpp:

```

// main.cpp
#include "student_manager.h"
#include <iostream>

```

```

int main() {
    StudentManager manager;

    manager.addStudent(Student(" 张三", "2023001", 85.5));
    manager.addStudent(Student(" 李四", "2023002", 92.0));
    manager.addStudent(Student(" 王五", "2023003", 78.5));

    manager.displayAll();
    manager.saveToFile("data.txt");

    manager.searchByName(" 李四");

    return 0;
}

```

编译命令:

方式 1: 一次性编译所有文件

```
g++ main.cpp student.cpp student_manager.cpp -o student_system
```

方式 2: 分别编译再链接

```
g++ -c student.cpp -o student.o
```

```
g++ -c student_manager.cpp -o student_manager.o
```

```
g++ -c main.cpp -o main.o
```

```
g++ student.o student_manager.o main.o -o student_system
```

Day 19: 简单项目管理与 CMake 实践

📖 19.1 基本知识点

- CMake 简介
- CMakeLists.txt 基本语法
- 多源文件编译
- 项目构建流程
- CMake 常用命令

🔗 19.2 学习资料

- CMake 官方教程

- CMake 中文入门实战

☑ 19.3 应知应会 Checklist

- ☐ 理解 CMake 的作用
- ☐ 能编写基本的 CMakeLists.txt
- ☐ 掌握 CMake 构建项目的流程
- ☐ 能使用 CMake 管理多文件项目

📝 19.4 小任务 使用 CMake 构建学生管理系统。

CMakeLists.txt:

```
# 指定 CMake 最低版本
cmake_minimum_required(VERSION 3.10)

# 项目名称和版本
project(StudentSystem VERSION 1.0)

# 指定 C++ 标准
set(CMAKE_CXX_STANDARD 11)
set(CMAKE_CXX_STANDARD_REQUIRED True)

# 添加可执行文件
add_executable(student_system
    main.cpp
    student.cpp
    student_manager.cpp
)

# 可选: 设置输出目录
set(EXECUTABLE_OUTPUT_PATH ${PROJECT_SOURCE_DIR}/bin)
```

构建步骤:

```
# 1. 创建构建目录
mkdir build
cd build

# 2. 生成构建文件
cmake ..
```

```
# 3. 编译项目
cmake --build .

# 或者使用 make (Linux/Mac)
make

# 4. 运行程序
./student_system # Linux/Mac
# 或
.\student_system.exe # Windows
```

更复杂的 CMakeLists.txt (可选) :

```
cmake_minimum_required(VERSION 3.10)
project(StudentSystem VERSION 1.0)

set(CMAKE_CXX_STANDARD 11)
set(CMAKE_CXX_STANDARD_REQUIRED True)

# 包含头文件目录
include_directories(${PROJECT_SOURCE_DIR}/include)

# 收集所有源文件
file(GLOB SOURCES "src/*.cpp")

# 添加可执行文件
add_executable(student_system ${SOURCES})

# 添加编译选项
if(MSVC)
    target_compile_options(student_system PRIVATE /W4)
else()
    target_compile_options(student_system PRIVATE -Wall -Wextra -pedantic)
endif()
```

Day 20-21: 综合项目实践 ①

■ **20.1 项目目标** 综合运用前三周所学知识，完成一个完整的面向对象项目。

🔗 20.2 项目选择

选项一：学生成绩管理系统（命令行版） 功能需求：

1. 添加学生信息（姓名、学号、成绩）
2. 删除学生信息
3. 修改学生成绩
4. 查询学生信息（按姓名或学号）
5. 显示所有学生
6. 计算平均分、最高分、最低分
7. 数据持久化（保存到文件、从文件加载）
8. 排序功能（按成绩排序）

类设计建议：

- Student 类：学生信息
- StudentManager 类：学生管理
- Menu 类：菜单系统
- FileHandler 类：文件操作

选项二：简易图书管理系统 功能需求：

1. 添加图书（书名、作者、ISBN、库存）
2. 删除图书
3. 借书/还书
4. 查询图书
5. 显示所有图书
6. 数据持久化

类设计建议：

- Book 类：图书信息
- Library 类：图书馆管理
- User 类：用户信息
- BorrowRecord 类：借阅记录

```
// menu.h
#ifndef MENU_H
#define MENU_H

#include "student_manager.h"
```

```

class Menu {
private:
    StudentManager manager;

    void showMenu();
    void handleAdd();
    void handleDelete();
    void handleModify();
    void handleSearch();
    void handleDisplay();
    void handleStatistics();
    void handleSort();
    void handleSave();
    void handleLoad();

public:
    void run();
};

#endif

```

```

// menu.cpp
#include "menu.h"
#include <iostream>
#include <limits>

void Menu::showMenu() {
    cout << "\n===== 学生成绩管理系统 =====" << endl;
    cout << "1. 添加学生" << endl;
    cout << "2. 删除学生" << endl;
    cout << "3. 修改成绩" << endl;
    cout << "4. 查询学生" << endl;
    cout << "5. 显示所有学生" << endl;
    cout << "6. 统计信息" << endl;
    cout << "7. 成绩排序" << endl;
    cout << "8. 保存数据" << endl;
    cout << "9. 加载数据" << endl;
}

```

```

    cout << "0. 退出系统" << endl;
    cout << "===== " << endl;
    cout << " 请选择操作: ";
}

void Menu::handleAdd() {
    string name, id;
    double score;

    cout << " 请输入姓名: ";
    cin >> name;
    cout << " 请输入学号: ";
    cin >> id;
    cout << " 请输入成绩: ";
    cin >> score;

    manager.addStudent(Student(name, id, score));
    cout << " 添加成功! " << endl;
}

void Menu::run() {
    int choice;

    while (true) {
        showMenu();
        cin >> choice;

        if (cin.fail()) {
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            cout << " 无效输入, 请重新选择! " << endl;
            continue;
        }

        switch (choice) {
            case 1: handleAdd(); break;
            case 2: handleDelete(); break;
            case 3: handleModify(); break;

```

```

        case 4: handleSearch(); break;
        case 5: handleDisplay(); break;
        case 6: handleStatistics(); break;
        case 7: handleSort(); break;
        case 8: handleSave(); break;
        case 9: handleLoad(); break;
        case 0:
            cout << " 感谢使用, 再见! " << endl;
            return;
        default:
            cout << " 无效选择, 请重新输入! " << endl;
    }
}
}

```

20.3 示例：学生成绩管理系统

20.4 项目要求

- ☐ 使用面向对象设计
- ☐ 合理的类层次结构
- ☐ 多文件组织
- ☐ 使用 CMake 构建
- ☐ 实现数据持久化
- ☐ 代码注释完整
- ☐ 用户交互友好

20.5 进阶挑战

- 使用 STL 容器优化数据存储
- 添加异常处理机制
- 实现更复杂的查询功能
- 使用智能指针管理内存
- 添加日志系统

Day 20-21.ex: 综合项目实践 ② - 控制台 Flappy Bird (OOP 版)

 **21.1 项目概述** 本项目旨在使用 C++ 面向对象编程 (OOP) 实现一个控制台版本的 Flappy Bird 小游戏。Flappy Bird 是一款经典的横版闯关游戏，玩家通过控制小鸟跳跃，

躲避不断移动的障碍物，并尽可能获取更高的分数。

本项目不仅考察 **C++ 面向对象设计能力**，还涉及 **控制台图形绘制、物理模拟、用户输入处理、游戏循环设计、碰撞检测和分数系统实现** 等综合编程技能。

通过本项目，希望同学们应掌握以下能力：

1. 将游戏元素抽象为类（小鸟、障碍物、游戏管理器）并进行封装。
 2. 实现小鸟的运动逻辑，包括 **重力模拟、跳跃操作**。
 3. 使用 **定时器和主循环** 控制游戏帧率。
 4. 通过 **字符界面** 实现游戏场景的绘制。
 5. 处理用户 **非阻塞输入**（空格键跳跃、退出游戏）。
 6. 实现障碍物的 **随机生成与移动**。
 7. 完成 **碰撞检测和分数统计**。
 8. 实现 **游戏结束判定和界面清屏刷新**。
-

🔗 21.2 功能与实现要点

1. 小鸟的跳跃与运动

- 使用 Bird 类封装小鸟的状态（纵坐标 y 和速度 velocity）。
- 小鸟每一帧受 **重力加速度** 作用向下运动。
- 当用户按下空格键时，小鸟获得一个向上的速度，实现跳跃效果。

2. 障碍物的生成与移动

- 使用 Obstacle 类封装障碍物属性(x 坐标、缺口位置 gapY、缺口大小 gapSize)。
- 每帧向左移动，当障碍物超出屏幕左侧时，重新生成在屏幕右侧并随机调整缺口位置。

3. 碰撞检测

- 小鸟与障碍物的碰撞判定：小鸟的坐标落在障碍物的非缺口区域即判定碰撞。
- 小鸟触底或飞出顶部同样判定游戏结束。

4. 分数统计

- 当小鸟成功通过障碍物时，分数加一。
- 分数在屏幕上方实时显示。

5. 控制台界面与刷新

- 使用 **字符界面绘制**游戏场景，包括小鸟和障碍物。
- 每帧刷新屏幕，保持游戏动态效果。

- 使用跨平台清屏方法 (Windows: `system("cls")`, Linux/Mac: `system("clear")`).

6. 非阻塞输入处理

- 实现非阻塞检测键盘输入，实现实时跳跃响应。
- 支持退出游戏操作（如按 Q 键退出）。

7. 游戏主循环设计

- 游戏主循环负责更新小鸟和障碍物状态、检测碰撞、绘制游戏界面，并处理用户输入。
 - 循环频率控制游戏速度，保证体验平滑。
-

☑ 21.3 类设计建议

Bird 类 负责封装小鸟的状态与行为：

```
class Bird {
public:
    int y;           // 小鸟纵坐标
    int velocity;    // 小鸟速度

    Bird(int startY);
    void jump();     // 跳跃操作
    void update();   // 更新物理状态（重力）
};
```

- `update()`：每一帧根据速度和重力更新 `y` 坐标
- `jump()`：改变速度实现跳跃

Obstacle 类 负责封装障碍物属性及行为：

```
class Obstacle {
public:
    int x;           // X 坐标
    int gapY;        // 缺口纵坐标
    int gapSize;     // 缺口大小

    Obstacle(int startX, int gapY, int gapSize);
    void update();   // 左移障碍物
};
```

```
bool isColliding(int birdX, int birdY) const; // 碰撞检测
};
```

- update(): 每帧向左移动
- isColliding(): 判断小鸟是否碰到障碍物

Game 类 负责管理游戏整体逻辑:

```
class Game {
private:
    Bird bird;
    std::vector<Obstacle> obstacles;
    int score;
    bool gameOver;

    void generateObstacle(); // 随机生成障碍物
    bool checkCollision() const; // 碰撞检测

public:
    Game();
    void run(); // 游戏主循环
    void update(); // 更新游戏状态
    void draw() const; // 绘制游戏界面
};
```

- run(): 主循环, 调用 update() 和 draw(), 处理用户输入
- update(): 更新小鸟位置、障碍物移动和碰撞检查
- draw(): 刷新控制台显示

21.4 实现提示与拓展

1. 物理模拟

- 可以设置常量 GRAVITY 控制重力加速度, JUMP_VELOCITY 控制跳跃高度。

2. 障碍物生成

- 随机缺口纵坐标, 保证可通过性
- 可设置不同难度 (如增加障碍物数量、减小缺口大小)

3. 控制台绘制优化

- 可使用 ANSI 转义序列移动光标，实现无闪烁刷新
- 或者使用第三方库如 ncurses (Linux) / conio.h (Windows)

4. 分数显示

- 分数随障碍物通过实时更新
- 可在游戏结束时显示最高分

5. 拓展挑战

- 添加不同速度的障碍物
- 添加小鸟动画效果（如翅膀扇动，可考虑使用字符的不断变换实现）
- 实现多关卡模式或速度递增模式
- 使用 EasyX(EasyX 中文官网) 或 SFML 等图形库实现更丰富的图形界面

这个版本中，所有游戏逻辑通过 **面向对象的类封装**，提高了 **模块化、可维护性和扩展性**。主函数只需实例化 Game 类并调用 run()，便可启动完整游戏流程。

如果大家在《C 语言三周入门》部分已经实现过 C 语言的控制台版本 Flappy Bird，可以尝试将其重构为**面向对象版本**，提升代码质量和结构。

五、进阶扩展模块（可选）

完成三周基础学习后，如有余力，可以选择以下进阶模块深入学习。这些模块涵盖了现代 C++ 的重要特性，是从入门到精通的必经之路。

学习建议：

- 这些模块**不强制学习**，但强烈推荐在掌握基础后逐步接触
- 建议按照 E1 → E3 → E2 → E4 的顺序学习
- 每个模块预计需要 2-3 天时间深入理解与实践

进阶模块概览

模块	主题	难度	推荐学习时机	重要程度
E1	泛型编程 (Templates)	★ ★ ★	完成第二周后	★ ★ ★ ★ ★
E2	智能指针 (Smart Pointers)	★ ★ ★ ★	完成第三周后	★ ★ ★ ★ ★

模块	主题	难度	推荐学习时机	重要程度
E3	STL 容器与算法	★★	完成第一周后	★★★★★
E4	Lambda 表达式与函数对象	★★★	学习 STL 之后	★★★★

E1: 泛型编程 (Templates)

📖 学习内容 核心知识点:

- 函数模板的定义与使用
- 类模板的定义与实例化
- 模板参数与类型推导机制
- 模板特化与偏特化
- 模板元编程初步

学习目标:

- 理解模板的编译期机制
- 能够编写泛型函数与泛型类
- 理解 STL 底层实现原理

推荐学习资料:

- 菜鸟教程 - C++ 模板
- cppreference - Templates

📁 实践任务 任务 1: 编写通用 swap() 函数模板

```
template <typename T>
void swap(T& a, T& b) {
    T temp = a;
    a = b;
    b = temp;
}
```

任务 2: 实现泛型 Vector<T> 类

- 支持任意类型的动态数组
- 实现 push_back(), pop_back(), size() 等方法
- 使用模板实现类型安全的容器

任务 3：模板特化

- 为特定类型（如 char*）编写特化版本
 - 理解全特化与偏特化的区别
-

E2: 智能指针 (Smart Pointers)

📖 学习内容 核心知识点:

- RAII (Resource Acquisition Is Initialization) 原则
- unique_ptr: 独占所有权的智能指针
- shared_ptr: 共享所有权的智能指针
- weak_ptr: 弱引用, 解决循环引用问题
- 自定义删除器

学习目标:

- 理解智能指针的生命周期管理机制
- 掌握不同智能指针的使用场景
- 能够避免内存泄漏和悬空指针

推荐学习资料:

- 微软教程 - C++ 智能指针
- cppreference - Smart pointers
- LearnCpp - Smart pointers

📌 实践任务 任务 1: 使用 unique_ptr 管理动态对象

```
std::unique_ptr<Student> stu = std::make_unique<Student>(" 张三", 20);  
// 自动释放内存, 无需手动 delete
```

任务 2: 使用 shared_ptr 实现引用计数

- 创建多个 shared_ptr 指向同一对象
- 观察引用计数的变化 (use_count())
- 理解最后一个 shared_ptr 销毁时才释放内存

任务 3: 解决循环引用问题

- 使用 weak_ptr 打破循环引用
 - 实现双向链表或图结构
-

E3: STL 容器与算法

学习内容 核心知识点:

- 序列容器: vector, list, deque, array
- 关联容器: set, map, multiset, multimap
- 无序容器: unordered_set, unordered_map
- 迭代器: 正向、反向、常量迭代器
- 算法库: sort, find, count, accumulate, transform 等

学习目标:

- 理解 STL 容器的内部实现与性能特点
- 能够选择合适的容器解决问题
- 熟练使用 STL 算法库

推荐学习资料:

- 菜鸟教程 - C++ STL 教程
- cppreference - Containers
- cppreference - Algorithms

实践任务 任务 1: 单词频率统计程序

```
std::map<std::string, int> wordCount;  
// 读取文本, 统计每个单词出现次数  
// 按频率排序输出
```

任务 2: 使用 vector 优化学生管理系统

- 将数组替换为 std::vector<Student>
- 使用 std::sort() 实现多种排序方式
- 使用 std::find_if() 实现条件查找

任务 3: 算法练习

- 使用 std::accumulate 计算平均分
- 使用 std::transform 批量数据转换
- 使用 std::copy_if 实现条件过滤

E4: Lambda 表达式与函数对象

学习内容 核心知识点:

- Lambda 表达式语法: [capture](params) -> return_type { body }

- 捕获方式：值捕获 [=]、引用捕获 [&]、混合捕获
- 函数对象 (Functor)
- std::function 与 std::bind
- Lambda 与 STL 算法的结合使用

学习目标:

- 能够灵活使用 Lambda 简化代码
- 理解闭包与捕获机制
- 在算法和回调中使用匿名函数

推荐学习资料:

- 微软教程 - C++ Lambda 表达式
- cppreference - Lambda expressions

实践任务 任务 1: 使用 Lambda 与 std::for_each

```
std::vector<int> nums = {1, 2, 3, 4, 5};
std::for_each(nums.begin(), nums.end(), [](int& n) {
    n *= 2; // 每个元素翻倍
});
```

任务 2: 自定义排序规则

```
std::sort(students.begin(), students.end(),
    [](const Student& a, const Student& b) {
        return a.getScore() > b.getScore();
    });
```

任务 3: 闭包与状态保持

- 使用 Lambda 捕获外部变量
- 实现计数器、累加器等有状态的函数对象

进阶学习路线建议

基础阶段 (三周)

↓

E3: STL 容器与算法 (2-3天)

↓

E1: 泛型编程 (3-4天)

↓

E4: Lambda 表达式 (2天)

↓

E2: 智能指针 (2-3天)

↓

实战项目：综合运用所有特性

提示：进阶模块的学习不必一次性完成，可以在实际项目中遇到需求时再深入学习对应模块。

六、学习建议与总结

6.1 学习建议

学习方法

1. 从 C 到 C++ 的思维转变

- 如果已完成《C 语言三周入门》，请认识到 C++ 不只是“带类的 C”，而是一种新的编程范式
- 面向对象编程强调**封装、继承、多态**，要学会用“对象”的视角思考问题
- 不要急于抛弃 C 的知识，C++ 完全兼容 C，可以平滑过渡

2. 理论与实践相结合

- 每天的知识点必须通过**代码实现**来验证理解
- 遇到不懂的概念，尝试写代码复现问题，观察程序行为
- 使用调试器（如 GDB、VS Debugger）单步调试，观察对象的创建与销毁

3. 循序渐进，不要跳跃

- C++ 的概念环环相扣：类 → 封装 → 继承 → 多态，每一步都很重要
- 如果某一天的内容没掌握，及时回顾补齐，避免影响后续学习
- 本教程设计为 21 天渐进式学习路径，建议按顺序完成

4. 重视代码规范与工程化思维

- 养成良好的代码风格：合理命名、适当注释、模块划分
- 从第三周开始学习**头文件分离、CMake 构建**，培养工程化意识
- 实际项目中，代码的可维护性和可扩展性至关重要

5. 善用学习资源

- 遇到问题优先查阅官方文档（如 cppreference.com）
- 利用在线编译器（如 Compiler Explorer、replit）快速验证代码
- 加入编程社区（如 Stack Overflow、GitHub、CSDN）交流学习

常见误区与注意事项

- **误区 1：**认为 C++ 只是多了几个关键字的 C

- **纠正**: C++ 引入了全新的编程思想 (OOP), 需要转变思维方式
- **误区 2**: 过早深入 STL、模板等高级特性
 - **纠正**: 先打好基础 (类、继承、多态), 再学习高级内容
- **误区 3**: 忽视内存管理
 - **纠正**: 虽然有智能指针, 但理解 RAII、拷贝构造、深浅拷贝仍然重要
- **误区 4**: 不写代码, 只看教程
 - **纠正**: 编程是实践学科, 必须动手敲代码

6.2 进阶方向

完成本教程后, 你已掌握 C++ 的核心基础知识。根据兴趣和职业发展方向, 可以选择以下进阶路径:

进阶路径建议

方向	推荐学习内容	应用场景
系统编程	Linux 系统编程、多线程 (pthread、C++11 thread)、网络编程	服务器开发、操作系统开发
游戏开发	C++ 图形库 (SDL、SFML)、游戏引擎 (Unreal Engine、Godot)	游戏引擎、3D 图形渲染
数据结构与算法	STL 深入、算法竞赛、LeetCode 刷题	编程竞赛、面试准备
嵌入式与硬件控制	Arduino C++、嵌入式 Linux、RTOS	物联网、机器人、智能硬件
现代 C++ 特性	C++11/14/17/20 新特性、智能指针、Lambda、移动语义	高性能计算、现代 C++ 项目开发
图形界面开发	Qt 框架、GTK+、wxWidgets	跨平台桌面应用开发
高性能计算与并行编程	OpenMP、CUDA、MPI	科学计算、深度学习加速
Robomaster 视觉开发	OpenCV、图像处理、自动瞄准算法	机器人视觉、自动化控制 (本战队重点)

推荐进阶书籍

- 《C++ Primer》(第 5 版): 经典 C++ 入门书籍, 系统全面
- 《Effective C++》: Scott Meyers 经典, 讲解 C++ 最佳实践
- 《深度探索 C++ 对象模型》: 理解 C++ 底层实现机制
- 《STL 源码剖析》: 侯捷著, 深入理解 C++ 标准库
- 《C++ Concurrency in Action》: 学习 C++ 多线程编程

6.3 总结

 **恭喜你完成 C++ 三周入门教程!**

通过 21 天的学习，你已经：

- ☒ **掌握了从 C 到 C++ 的语法演进**，理解了面向对象编程的核心思想
 - ☒ **学会了类与对象的定义与使用**，能够封装数据与行为
 - ☒ **理解了继承、多态、抽象类**，能够设计灵活的类层次结构
 - ☒ **掌握了文件操作与模块化编程**，能够组织多文件项目
 - ☒ **使用 CMake 构建工程**，具备了基本的工程化开发能力
 - ☒ **完成了综合项目实践**，能够独立设计与实现一个完整的 C++ 程序
-

下一步行动

1. **巩固基础**：回顾每天的 Checklist，确保所有知识点都已掌握
2. **拓展项目**：在学生管理系统或 Flappy Bird 基础上增加新功能
3. **学习 STL**：开始学习 C++ 标准模板库，提高代码效率
4. **参与实战**：加入战队项目开发，将所学应用到机器人视觉等实际场景
5. **持续学习**：编程是终身学习的过程，保持好奇心和探索精神

“学习编程不是为了成为程序员，而是为了更好地理解和创造这个世界。”

 **继续加油，期待在后续的学习和实战中看到你的成长!**